

Computational Frameworks

Streaming

OUTLINE

- ① Introduction to the streaming model (Part 1)
 - Warm-up: finding the majority element
- ② Sampling (Part 2)
 - Uniform sampling with reservoir sampling
 - Finding frequent items with sticky sampling
- ③ Sketching (Part 3)
 - Counting distinct elements with probabilistic counting
 - Estimating moments with AMS sketch

Introduction to the streaming model

Dealing with volume and velocity

Monitoring huge and rapidly changing streams of data:

- **Scenarios:** (sensor) network traffic, transaction logs, online trading, auctions, analyzing physical (e.g., meteorological, astronomical) events.
- **Data analysis** must happen **on the fly** using limited memory (cannot store the data stream for offline processing).
- Input data are to be processed as a continuous stream (**no random access to data**).

Some applications

Network management:

- **Stream:** packets routed through a router.
- **Task:** gather traffic statistics (e.g., average number of connections/second to same IP address)

Online auctions:

- **Stream:** online auctions generate data on objects on sale, bids, ...
- **Tasks:** statistical estimation of auction data (e.g., average selling price over the items sold by each seller, highest bid in a given period of time, ...)

Some applications

Internet of Things:

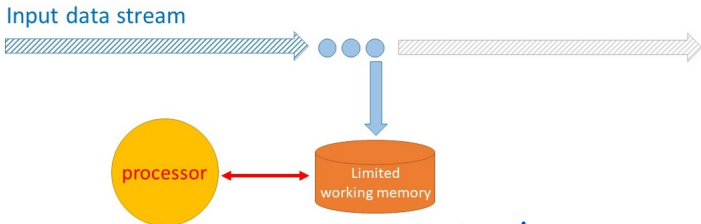
- **Stream:** data generated by thousands/millions of sensors.
- **Tasks:** learn from data, outlier detection, gather statistics,...

Sequential disk accesses:

- **Stream:** data from a massive file on secondary storage (sequential access rates are higher than random access),
- **Tasks:** data processing of data in the file. In this case it is also possible to sequentially scan the same file twice or even more times (**multi-pass**). In this scenario you can afford multiple passes but cannot afford (too many) random accesses to the data

Streaming Model

- (Sequential) machine with **limited amount of working memory**.
- Input provided as a **continuous (one-way) stream**.



Data may arrive very fast, hence they must be consumed very fast

Streaming Model

The Model

- **Input stream** $\Sigma = x_1, x_2, \dots, x_n$ accessed/processed sequentially
- **Metric 1:** Size s of the working memory (**aim:** $s \ll n$)
- **Metric 2:** Number p of sequential passes over Σ (**aim:** $p = 1$)
- **Metric 3:** Processing time per item T (**aim:** $T = O(1)$)

Algorithm Design Techniques

- **Approximate solutions** (exact ones may require linear space)
- Maintain **lossy summary** of Σ via a **synopsis** data structure (e.g., **random sample**, **hash-based sketch**)

Typical data analysis tasks

- Number of distinct items.
- Frequent items.
- Useful statistics:
e.g., frequency moments, quantiles, histograms, etc.
- Optimization and graph problems:
e.g., clustering, triangle counting

Warm-up:
finding the majority element in a stream

Warm-up: finding the majority element

Majority problem: given a stream $\Sigma = x_1, x_2, \dots, x_n$, detect if there exists a value x that appears in at least $> n/2$ elements of Σ and return it.

Examples:

- In $\Sigma = A, B, A, C, A, D, A, A$, the majority element is A .
- In $\Sigma = A, B, C, D, D, E, E$, a majority element does not exist.

Observation. In the standard sequential setting, we can sort and check for the majority in $O(n \log n)$ time using sorting ($O(n)$ time in expectation with a hash table) and $\Theta(n)$ space.

Boyer-Moore algorithm

The Boyer-Moore algorithm maintains 2 integers *cand* and *count*:

- *cand* contains a candidate majority element;
- *count* is a counter;

The algorithm returns a majority element if $\exists$ one, but it is unable to say whether a majority element exists. For that, you need a second pass

Initialization: *cand* $\leftarrow$ null and *count* $\leftarrow$ 0.

For each x_i in Σ :

```
if count = 0 then { cand  $\leftarrow$   $x_i$ ; count  $\leftarrow$  1; }  
else {  
  if cand =  $x_i$  then count  $\leftarrow$  count + 1  
  else count  $\leftarrow$  count - 1  
}
```

At the end of the algorithm, return *cand*

We will show that

If a majority element exists $\Rightarrow$ *cand* = majority element

Example

$x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9$

$\Sigma = A, A, A, C, C, B, B, A, A$

Step	Cand	Count
0	null	0
1	A	1
2	A	2
3	A	3
4	A	2
5	A	1
6	A	0
7	B	1
8	B	0
9	A	1

Candidate = A

Indeed, A is the majority element

Example

$\Sigma = A, A, A, C, C, C, B, B, B$

Step	Cand	Count
0	null	0
1	A	1
2	A	2
3	A	3
4	A	2
5	A	1
6	A	0
7	B	1
8	B	2
9	B	3

Candidate = B but there is no majority element

Warm-up: finding the majority element

Theorem

Given a stream Σ which contains a majority element m , the Boyer-Moore algorithm returns m using:

- working memory of size $s = O(1)$;
- $p = 1$ passes;
- $T = O(1)$ time per element.

Proof : Trivial for what concerns the 3 bullets
We only need to argue correctness carefully
Namely, we need to show that if there is a majority
element, then what the algorithm returns
is indeed this majority element

Suppose that we observed $x_1, x_2 \dots x_t$ $t \in [0, n]$
($t=0$: beginning of the processing before x_1 arrives)
What do cand and count represent?

- * There are count occurrences of cand in $x_1, x_2 \dots x_t$ (but they may not be all of the occurrences of cand !)
- * Besides the count occurrences of cand , the remaining $t - \text{count}$ elements form $(t - \text{count})/2$ pairs (e_1, e_2) with $e_1 \neq e_2$

E.g. $t=5$ in the first example

Count=1, cand=A, remaining elements: AAC $\Rightarrow$

- * 1 occurrence of A

- * $t-1=4$ elements forming 2 pairs (A,c) (A,c)
each with distinct elements

EXERCISE: show that the invariant holds
at every step

let cand_n the candidate returned at the end
i.e., the value cand after processing x_n

If Σ has a majority element, then it is cand_n

WHY ?

By contradiction, suppose that the majority element is $y \neq \text{cand}_n$

By the invariant we know that

- * cand_n occurs count times in Σ ($\text{count} \geq n$)

- * there are $(n - \text{count})/2$ pairs of distinct elements besides the count occurrences of cand_n

$y \neq \text{count}_n$ can occur at most $(n - \text{count})/2$ times

$$\frac{n - \text{count}}{2} \leq \frac{n}{2} \quad \text{since we always have } \text{count} \geq 0$$

$\Rightarrow y$ cannot be the majority element

□

Observations

- The Boyer-Moore algorithm always returns an element in *can**d*, even when there is no majority.
- A second pass over the stream can be used to check if *can**d* is a majority element: just count the occurrences of *can**d* and check if $> n/2$.
- If only one pass is allowed, we need to relax the assumptions on the output, e.g. by providing an element that appears *approximately* $\geq n/2$ times in the stream (we will see this in Part 2).

Exercise

Let $\Sigma = x_1, x_2, \dots, x_n$ be a stream of n distinct integers in $[1, n+1]$. Design a streaming algorithm that find the missing number using $O(1)$ -size working memory, 1 pass, and $O(1)$ time per element.